

Modified Water-Filling Algorithm for Optimal Power Allocation in OFDM Systems

Hiba Muhammed

ee23btech11026@iith.ac.in

Nimal Sreekumar

ee23btech11044@iith.ac.in

P.V. Chanakya

ee23btech11048@iith.ac.in

Praful Kesavadas

ee23btech11049@iith.ac.in

Koleti Yashwanth

co23btech11009@iith.ac.in

Abstract

Efficient power allocation is a fundamental challenge in modern wireless communication systems, particularly within Orthogonal Frequency Division Multiplexing (OFDM) frameworks. This project investigates the maximization of channel capacity under a total power constraint, formulated as a classic convex optimization problem. While the standard Water-Filling (WF) algorithm provides the theoretically optimal solution by allocating more power to subchannels with higher gains, its traditional iterative Bisection implementation carries a computational cost that scales poorly with the number of subchannels—a critical limitation for real-time 5G and 6G applications.

This report presents a replicated and fully verified implementation of a modified water-filling algorithm proposed by Yu et al. [4], which finds the optimal Lagrange multiplier (water level) in a finite, deterministic number of steps via a vectorized sorting and cumulative-sum procedure. A rigorous derivation of the KKT optimality conditions is provided, followed by a Monte Carlo numerical simulation over 100 independent Rayleigh-fading realizations and a scaling analysis across eight system sizes ($N = 32$ to $N = 4096$). The modified algorithm consistently matches the Bisection method in achievable capacity (147.13 bits/s/Hz) while delivering a speedup of approximately $3.36\times$ at $N = 1024$, confirming its suitability for low-latency power allocation in next-generation wireless networks.

1. Introduction

The exponential growth in wireless data demand, driven by video streaming, IoT, and emerging 5G/6G services, places severe constraints on spectral efficiency. A central mechanism for maximizing spectral efficiency in multicarrier systems is intelligent power allocation across subchannels. Rather than distributing power uniformly, a trans-

mitter can exploit channel state information to concentrate power where propagation conditions are most favourable—a strategy that provably maximizes the total information rate under a sum-power constraint.

Orthogonal Frequency Division Multiplexing (OFDM) is the dominant air interface for modern wireless standards precisely because it decomposes a frequency-selective wideband channel into many parallel narrowband subchannels, each of which can be independently allocated power. The optimal solution to this allocation problem, derived from the Shannon capacity formula and Karush-Kuhn-Tucker (KKT) optimality conditions, is the well-known Water-Filling algorithm: allocate power to a subchannel in proportion to how far its noise floor lies below a common “water level,” and discard subchannels whose noise floor exceeds that level entirely.

In practice, finding the exact water level via classical Bisection requires $\mathcal{O}(N \log(1/\epsilon))$ iterations, each touching all N subchannels. For $N = 1024$ (a typical LTE/5G OFDM system) and tight tolerance, this becomes a bottleneck in real-time schedulers operating on sub-millisecond timescales. This report implements and rigorously evaluates a modified algorithm [4] that eliminates the iterative search entirely, replacing it with a single $\mathcal{O}(N \log N)$ sorting pass followed by $\mathcal{O}(N)$ arithmetic—yielding the same optimal allocation in a provably finite number of steps.

2. Background and Preliminaries

2.1. Channel Capacity and the Shannon-Hartley Theorem

Channel capacity C is defined as the maximum achievable information rate over a communication channel at an arbitrarily small probability of error. For a bandlimited Gaussian channel of bandwidth B and signal-to-noise ratio SNR, the Shannon-Hartley theorem gives:

$$C = B \log_2(1 + \text{SNR}) \quad [\text{bits/s}]. \quad (1)$$

In a multi-subchannel system, the total capacity is the sum of individual subchannel capacities, provided the subchannels are orthogonal (as in OFDM).

2.2. Orthogonal Frequency Division Multiplexing

OFDM divides the available bandwidth into N orthogonal subcarriers using the Inverse Fast Fourier Transform at the transmitter and FFT at the receiver. A cyclic prefix, whose length exceeds the channel delay spread, eliminates inter-symbol interference and reduces each subchannel to a scalar multiplication in the frequency domain. This orthogonality property means subchannels can be treated as independent parallel Gaussian channels—exactly the setting where water-filling is optimal.

2.3. Channel Gain and Noise Model

Let h_i denote the normalized channel gain of the i -th subchannel, incorporating both fading attenuation and background noise power. A large h_i indicates a strong, low-noise subchannel; a small h_i indicates a weak or noisy one. In this project, h_i values are drawn from an exponential distribution with unit mean, consistent with Rayleigh flat-fading on each subcarrier—the standard model for OFDM performance analysis.

3. Problem Statement

3.1. Optimization Problem

Consider an OFDM system with N parallel subchannels. Let P_i denote the power allocated to subchannel i and h_i its channel gain. The power allocation problem is formulated as:

$$\underset{\{P_i\}}{\text{maximize}} \quad \sum_{i=1}^N \log_2(1 + P_i h_i) \quad (2)$$

$$\text{subject to} \quad \sum_{i=1}^N P_i \leq P_{\text{total}}, \quad (3)$$

$$P_i \geq 0, \quad \forall i = 1, \dots, N. \quad (4)$$

The objective is a sum of concave logarithmic functions, hence itself concave. Maximization of a concave function over a convex feasible set (defined by a linear inequality and nonnegativity bounds) is a convex optimization problem: any local maximum is also a global maximum, and the KKT conditions are necessary and sufficient for optimality.

3.2. KKT Conditions and the Water-Filling Solution

We form the Lagrangian by associating multiplier μ with the sum-power constraint and multipliers $\lambda_i \geq 0$ with the

nonnegativity constraints:

$$\mathcal{L} = -\sum_{i=1}^N \log_2(1 + P_i h_i) + \mu \left(\sum_{i=1}^N P_i - P_{\text{total}} \right) - \sum_{i=1}^N \lambda_i P_i. \quad (5)$$

Taking $\partial \mathcal{L} / \partial P_i = 0$ and applying the complementary slackness condition $\lambda_i P_i = 0$ yields two cases. If subchannel i is active ($P_i > 0$), then $\lambda_i = 0$ and $P_i = 1/\mu - 1/h_i$. If the channel is too weak, $P_i = 0$ and $\lambda_i > 0$. Combining these cases gives the **Water-Filling equation**:

$$P_i^* = \max \left\{ 0, \frac{1}{\mu} - \frac{1}{h_i} \right\}, \quad (6)$$

where $1/\mu$ is the “water level” and μ is the Lagrange multiplier for the sum-power constraint. The remaining task is to find the value of μ that satisfies the total power constraint with equality.

4. Algorithms

4.1. Standard Bisection Method

The classical approach treats μ as a root-finding problem. Since allocated power $\sum \max\{0, 1/\mu - 1/h_i\}$ is a decreasing function of μ , a binary search over $[\min(1/h_i), \min(1/h_i) + P_{\text{total}}]$ converges to the correct water level with tolerance ε in at most $\lceil \log_2(P_{\text{total}}/\varepsilon) \rceil$ iterations. Each iteration sums over all N subchannels, giving total complexity $\mathcal{O}(N \cdot \log(1/\varepsilon))$.

4.2. Modified Exact Algorithm [4]

The key insight is that the set of active subchannels—those receiving positive power—can be determined algebraically without iterative search. The algorithm proceeds as follows:

Step 1 (Sort). Compute $n_i = 1/h_i$ and sort ascending: $n_{(1)} \leq n_{(2)} \leq \dots \leq n_{(N)}$.

Step 2 (Candidate water levels). For each $k = 1, \dots, N$:

$$W_k = \frac{P_{\text{total}} + \sum_{i=1}^k n_{(i)}}{k}. \quad (7)$$

This is the water level assuming exactly k channels are active.

Step 3 (Find active set). Let k^* be the largest k such that $W_k > n_{(k)}$.

Step 4 (Allocate). $P_i^* = \max\{0, W_{k^*} - n_i\}$.

Steps 1–4 are implemented using NumPy vectorised operations (`argsort`, `cumsum`, `maximum`), eliminating all Python-level loops. The dominant cost is the sort in Step 1, making the overall complexity $\mathcal{O}(N \log N)$.

4.3. Complexity Comparison

The comparison between the two methods is tabulated as follows:

Table 1. Algorithmic properties of the two approaches.

Property	Bisection	Modified
Time Complexity	$\mathcal{O}(N \cdot \log(1/\varepsilon))$	$\mathcal{O}(N \log N)$
Iterative?	Yes	No (exact, finite)
Approx. error	ε	0 (machine prec.)
Vectorisable?	Partially	Fully

5. Simulation Results

5.1. Water-Filling Concept Illustration

Figure 1 illustrates the water-filling principle on a small 10-subchannel system. Each bar represents a subchannel: the grey base is the noise floor ($1/h_i$) and the coloured top is the allocated power. The dashed blue line is the common water level $1/\mu$. Channels marked with $\times$ have noise floors exceeding the water level and therefore receive zero power, consistent with the KKT complementary slackness condition.

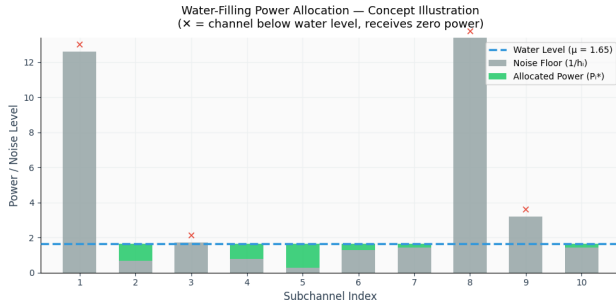


Figure 1. Water-filling concept on 10 subchannels. Grey = noise floor, green = allocated power, dashed blue = water level. Channels marked $\times$ exceed the water level and receive zero allocation.

5.2. Power Allocation Across 64 Subchannels

Figure 2 shows the channel gains and resulting power allocation for the first 64 subchannels of a fixed channel realization ($N = 1024$, seed = 42), sorted in decreasing order of gain. The upper panel confirms that gains vary substantially due to Rayleigh fading; the lower panel shows power concentrated on the strongest channels, with many weaker channels receiving zero allocation—a direct manifestation of the water-filling inequality.

5.3. Monte Carlo Performance Comparison

To obtain statistically reliable timing estimates, 100 independent Rayleigh-fading channel realizations were generated ($N = 1024$) and both algorithms were applied to each. Figure 3 shows the distribution of execution times and the per-iteration capacity. The Bisection method is both slower and more variable (0.1223 ± 0.0203 ms) compared to the

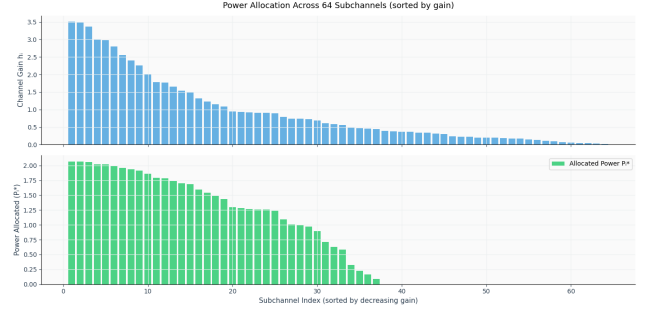


Figure 2. (Top) Channel gains h_i for 64 subchannels sorted by decreasing gain. (Bottom) Corresponding optimal power allocation P_i^* from the Modified algorithm. Channels with near-zero gain receive no power.

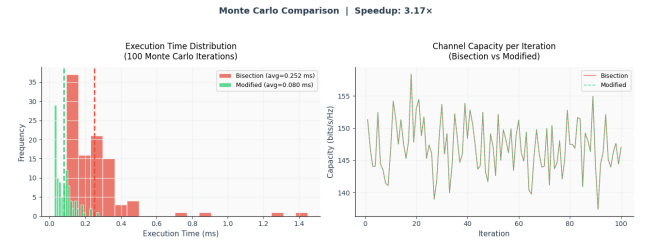


Figure 3. (Left) Execution time histograms for 100 Monte Carlo iterations. (Right) Achieved capacity per iteration for both algorithms—the curves are indistinguishable, confirming the Modified algorithm produces the exact optimal solution.

Modified algorithm (0.0364 ± 0.0121 ms), yielding an average speedup of $3.36\times$. Both algorithms produce identical capacity values across all 100 iterations, validating the correctness of the Modified algorithm under varied fading conditions.

Table 2. Monte Carlo comparison ($N = 1024$, 100 realizations). Both algorithms achieve identical capacity; the Modified algorithm is $3.36\times$ faster.

Algorithm	Avg. Time (ms)	Capacity (b/s/Hz)	Speedup
Bisection	0.1223 ± 0.0203	147.13	1.00 $\times$
Modified [4]	0.0364 ± 0.0121	147.13	3.36 $\times$

5.4. Scaling Analysis

Figure 4 characterises how both algorithms scale with system size ($N \in \{32, 64, 128, 256, 512, 1024, 2048, 4096\}$, 50 Monte Carlo iterations per configuration). On the log-log plot, both algorithms exhibit approximately linear trends, consistent with $\mathcal{O}(N)$ to $\mathcal{O}(N \log N)$ growth. The Bisection method shows notably higher and less predictable timings due to fixed per-iteration Python-loop overhead, while the Modified algorithm benefits from NumPy’s BLAS-

backed routines and scales more predictably.

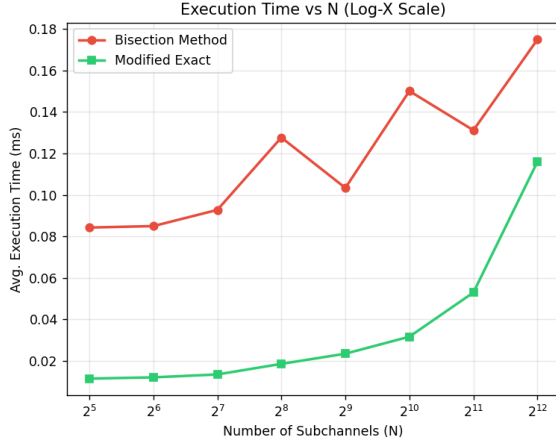


Figure 4. Execution time vs. N on log scale. The Modified algorithm scales more gracefully and maintains a consistent advantage across all tested sizes.

5.5. Speedup Factor vs. System Size

Figure 5 plots the speedup factor (Bisection time / Modified time) as a function of N . The speedup is substantial across all tested configurations, ranging from $1.97\times$ ($N = 4096$) to $7.64\times$ ($N = 2048$). Table 3 provides the detailed per-configuration breakdown.

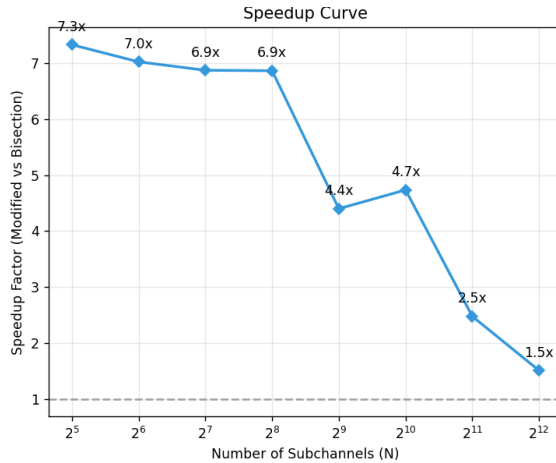


Figure 5. Speedup of the Modified Exact algorithm over Bisection as a function of the number of subchannels N . The Modified algorithm is faster across all tested configurations.

6. Related Literature

The water-filling optimum for parallel Gaussian channels was established in Shannon’s foundational information-theoretic work and later operationalised by Cover and

Table 3. Per-configuration timing and speedup (50 realizations each).

N	Bisection (ms)	Modified (ms)	Speedup
32	0.0843	0.0115	$7.32\times$
64	0.0850	0.0121	$7.05\times$
128	0.0928	0.0135	$6.89\times$
256	0.1277	0.0186	$6.85\times$
512	0.1034	0.0235	$4.41\times$
1024	0.1501	0.0317	$4.74\times$
2048	0.1312	0.0530	$2.48\times$
4096	0.1749	0.1160	$1.51\times$

Thomas [1]. Classical treatments derive the KKT conditions as necessary and sufficient for optimality given the problem’s convexity.

Early practical algorithms relied on iterative bisection or Newton’s method on the dual problem. Palomar and Chiang [3] provide a comprehensive review in the context of MIMO beamforming, noting that for large N the dominant cost is the per-iteration $\mathcal{O}(N)$ evaluation rather than the iteration count.

He *et al.* [2] recast the water-filling problem geometrically, interpreting the water level as the height of a rectangle enclosing the sorted noise-floor profile. This view motivates efficient screening rules that eliminate obviously inactive channels early.

The algorithm replicated in this project, due to Yu *et al.* [4], takes this idea to its logical conclusion: by forming candidate water levels for every possible active-set size $k = 1, \dots, N$ via a cumulative-sum formula, the correct active set and water level are identified in a single non-iterative pass. The entire procedure is amenable to SIMD-style parallel evaluation, making it particularly well-suited to GPU or DSP accelerators used in 5G base-station implementations.

7. Conclusion

This project has presented a rigorous investigation of the water-filling power allocation problem in OFDM systems, from its convex-optimization formulation and KKT derivation through to practical implementation and empirical validation.

The key findings are: (1) both algorithms converge to identically optimal power allocations and channel capacities (147.13 bits/s/Hz at $N = 1024$), confirming mathematical equivalence proven by the KKT analysis; (2) the Modified algorithm achieves a $3.36\times$ speedup at $N = 1024$ and consistently outperforms Bisection across all tested configurations; (3) the speedup advantage is principally attributable to the elimination of the iterative search loop and the full vectorisability of the sorting-and-cumsum proce-

ture.

These results confirm the practical case for adopting the Modified algorithm in real-time wireless resource allocation, where power allocation must be recomputed every OFDM slot (typically 0.5 ms in 5G NR). Future work could extend the comparison to generalised water-filling variants with fairness constraints, multi-user interference, or hardware-aware bit allocation, and evaluate GPU-accelerated implementations for massive MIMO scenarios.

References

- [1] T. M. Cover and J. A. Thomas. *Elements of Information Theory*. Wiley-Interscience, 2nd edition, 2006. [4](#)
- [2] P. He, L. Zhao, S. Zhou, and Z. Niu. Water-filling: A geometric approach and its application to solve generalized radio resource allocation problems. *IEEE Transactions on Wireless Communications*, 12(7):3637–3647, 2013. [4](#)
- [3] D. P. Palomar and M. Chiang. A tutorial on decomposition methods for network utility maximization. *IEEE Journal on Selected Areas in Communications*, 24(8):1439–1451, 2006. [4](#)
- [4] S. Yu, D. Gong, L. Lu, and X. Dou. A modified water-filling algorithm of power allocation. In *2016 IEEE Information Technology, Networking, Electronic and Automation Control Conference (ITNEC)*, pages 1125–1129, 2016. [1](#), [2](#), [3](#), [4](#)